

Martin Dalla Pozza

Informe de Análisis de APK Maliciosa **Asignatura: Telefonía Móvil**

Resumen:

Se realizó un análisis estático de la aplicación Android malicious.apk con el objetivo de determinar su naturaleza. Utilizando herramientas especializadas en Kali Linux, se examinaron la firma digital, los permisos solicitados, los componentes declarados en el manifiesto y el código descompilado. Los hallazgos revelan la presencia de clases pertenecientes al framework Metasploit, una amplia gama de permisos peligrosos y componentes diseñados para garantizar la persistencia en el dispositivo. Se concluye que la APK es un troyano de acceso remoto (RAT) generado con Meterpreter, una herramienta de pentesting.

```

(kali@kali)-[~]
$ apksigner verify --verbose --print-certs /home/kali/malicious.apk
Verifies
Verified using v1 scheme (JAR signing): true
Verified using v2 scheme (APK Signature Scheme v2): true
Verified using v3 scheme (APK Signature Scheme v3): true
Verified using v4 scheme (APK Signature Scheme v4): false
Verified for SourceStamp: false
Number of signers: 1
Signer #1 certificate DN: CN=Ruben Capria, OU=Oracle VirtualBox, O=Oracle, L=Barcelona, ST=Barcelona, C=BC
Signer #1 certificate SHA-256 digest: fe2763a5a33306d611d5bd82e528d75aedd621d736f3f918740e054ce615f7b4
Signer #1 certificate SHA-1 digest: ff4e93f5357a2b41f5db2434b75cb02aa7e30de5
Signer #1 certificate MD5 digest: 10dcfbfeba298705883eb26ca62a818f
Signer #1 key algorithm: RSA
Signer #1 key size (bits): 2048
Signer #1 public key SHA-256 digest: 4b76f967945c87b649a33516126dc3d1b34f588a897505604f0e6e4f7409f5b7
Signer #1 public key SHA-1 digest: eeeb8a0f7b1851e99b7254e8ef833df072170478
Signer #1 public key MD5 digest: d47d8b0080ba728a073ca44abcd51e1b

```

```

(kali@kali)-[~]
$ aapt dump badging /home/kali/malicious.apk | grep permission
uses-permission: name='android.permission.INTERNET'
uses-permission: name='android.permission.ACCESS_WIFI_STATE'
uses-permission: name='android.permission.CHANGE_WIFI_STATE'
uses-permission: name='android.permission.ACCESS_NETWORK_STATE'
uses-permission: name='android.permission.ACCESS_COARSE_LOCATION'
uses-permission: name='android.permission.ACCESS_FINE_LOCATION'
uses-permission: name='android.permission.READ_PHONE_STATE'
uses-permission: name='android.permission.SEND_SMS'
uses-permission: name='android.permission.RECEIVE_SMS'
uses-permission: name='android.permission.RECORD_AUDIO'
uses-permission: name='android.permission.CALL_PHONE'
uses-permission: name='android.permission.READ_CONTACTS'
uses-permission: name='android.permission.WRITE_CONTACTS'
uses-permission: name='android.permission.WRITE_SETTINGS'
uses-permission: name='android.permission.CAMERA'
uses-permission: name='android.permission.READ_SMS'
uses-permission: name='android.permission.WRITE_EXTERNAL_STORAGE'
uses-permission: name='android.permission.RECEIVE_BOOT_COMPLETED'

```

```

uses-permission: name='android.permission.SET_WALLPAPER'
uses-permission: name='android.permission.READ_CALL_LOG'
uses-permission: name='android.permission.WRITE_CALL_LOG'
uses-permission: name='android.permission.WAKE_LOCK'
uses-permission: name='android.permission.REQUEST_IGNORE_BATTERY_OPTIMIZATIONS'
uses-permission: name='android.permission.READ_EXTERNAL_STORAGE'
uses-implies-permission: name='android.permission.READ_EXTERNAL_STORAGE' reason='requested WRITE_EXTERNAL_STORAGE'
uses-implies-feature: name='android.hardware.location' reason='requested android.permission.ACCESS_COARSE_LOCATION permission, and requested android.permission.ACCESS_FINE_LOCATION permission'
uses-implies-feature: name='android.hardware.location.gps' reason='requested android.permission.ACCESS_FINE_LOCATION permission, and targetSdkVersion < 21'
uses-implies-feature: name='android.hardware.location.network' reason='requested android.permission.ACCESS_COARSE_LOCATION permission, and targetSdkVersion < 21'
uses-implies-feature: name='android.hardware.telephony' reason='requested a telephony permission'
uses-implies-feature: name='android.hardware.wifi' reason='requested android.permission.ACCESS_WIFI_STATE permission, and requested android.permission.CHANGE_WIFI_STATE permission'

```

```

(kali㉿kali)-[~]
$ strings /home/kali/malicious.apk | grep -i "meterpreter\|ahmyth\|reverse_tcp\|4444\|192.168"

(kali㉿kali)-[~]
$ apktool d /home/kali/malicious.apk -o salida
I: Using Apktool 2.7.0-dirty on malicious.apk
I: Loading resource table ...
I: Decoding AndroidManifest.xml with resources ...
I: Loading resource table from file: /home/kali/.local/share/apktool/framework/1.apk
I: Regular manifest package ...
I: Decoding file-resources ...
I: Decoding values */* XMLs ...
I: Baksmaling classes.dex ...
I: Copying assets and libs ...
I: Copying unknown files ...
I: Copying original files ...

```

```

(kali㉿kali)-[~]
$ cd salida

(kali㉿kali)-[~/salida]
$ cat AndroidManifest.xml | grep -i "service\|receiver\|boot"
<uses-permission android:name="android.permission.RECEIVE_BOOT_COMPLETED" />
<receiver android:label="MainBroadcastReceiver" android:name=".MainBroadcastReceiver">
  <action android:name="android.intent.action.BOOT_COMPLETED" />
</receiver>
<service android:exported="true" android:name=".MainService" />

(kali㉿kali)-[~/salida]
$

(kali㉿kali)-[~/salida]
$ grep -r "192.168" smali/
grep -r "4444" smali/
grep -r "http" smali/
smali/com/metasploit/stage/b.smali:    const-string v5, "http"
smali/com/metasploit/stage/Payload.smali:    const-string v2, "https"

```

Introducción:

El análisis de aplicaciones Android sospechosas es fundamental para identificar malware que pueda comprometer la privacidad y seguridad de los usuarios. En este trabajo se analiza una muestra proporcionada, aplicando técnicas de análisis estático que permiten examinar la estructura interna de la APK sin ejecutarla. El objetivo es determinar si la aplicación es legítima o si ha sido generada con herramientas como Meterpreter (Metasploit) o AhMyth.

Metodología:

El análisis se llevó a cabo en un entorno controlado con sistema operativo Kali Linux, utilizando las siguientes herramientas:

apksigner: verificación de firmas y certificados.

aapt: extracción de permisos y metadatos del manifiesto.

apktool: descompilación de recursos y código smali.

strings: búsqueda de cadenas de texto en el binario.

grep: filtrado de resultados.

Resultados:

Verificación de firma

Se ejecutó el comando:

apksigner verify --verbose --print-certs /home/kali/malicious.apk

Resultado:

La APK utiliza los esquemas de firma v1, v2 y v3 (v4 no presente).

Certificado del firmante: CN=Ruben Capria, OU=Oracle VirtualBox, O=Oracle, L=Barcelona, ST=Barcelona, C=BC.

Algoritmo: RSA con clave de 2048 bits.

Interpretación:

La firma es válida y no presenta anomalías por sí misma. El nombre del firmante no es el típico "Android Debug", pero tampoco corresponde a una entidad conocida; sin embargo, esto no es concluyente.

Análisis de permisos:

Mediante `aapt dump badging malicious.apk | grep permission` se obtuvo la lista de permisos solicitados. A continuación se muestran los más relevantes (todos ellos considerados peligrosos):

Permiso	Descripción	Nivel de riesgo
ACCESS_FINE_LOCATION	Ubicación precisa (GPS)	Alto
ACCESS_COARSE_LOCATION	Ubicación aproximada (red)	Alto
CAMERA	Acceso a la cámara	Alto
RECORD_AUDIO	Grabación de audio	Alto
READ_PHONE_STATE	Leer estado del teléfono	Alto
CALL_PHONE	Realizar llamadas	Alto
SEND_SMS	Enviar mensajes SMS	Alto
RECEIVE_SMS	Recibir SMS	Alto
READ_SMS	Leer SMS	Alto
READ_CONTACTS	Leer contactos	Alto
WRITE_CONTACTS	Modificar contactos	Alto
READ_CALL_LOG	Leer registro de llamadas	Alto
WRITE_CALL_LOG	Modificar registro de llamadas	Alto
RECEIVE_BOOT_COMPLETED	Ejecutarse al arrancar el dispositivo	Medio (persistencia)

Además, se incluyen permisos como:

INTERNET, ACCESS_NETWORK_STATE, WAKE_LOCK REQUEST_IGNORE_BATTERY_OPTIMIZATIONS

Entre otros, que facilitan la comunicación remota y la permanencia en segundo plano.

Interpretación:

La gran cantidad de permisos sensibles es típica de un troyano de acceso remoto (RAT). Una aplicación legítima no necesitaría acceder a cámara, micrófono, ubicación, SMS, contactos y llamadas simultáneamente.

Descompilación y análisis de componentes:

Se descompiló la APK con apktool d malicious.apk -o salida, obteniendo la estructura de archivos. Luego se examinó el AndroidManifest.xml y el código smali.

Componentes en el manifiesto:

Del archivo AndroidManifest.xml se extrajeron los siguientes elementos:

```
<uses-permission android:name="android.permission.RECEIVE_BOOT_COMPLETED"/>
<receiver android:label="MainBroadcastReceiver" android:name=".MainBroadcastReceiver">
  <action android:name="android.intent.action.BOOT_COMPLETED"/>
</receiver>
<service android:exported="true" android:name=".MainService"/>
```

MainBroadcastReceiver:

Escucha el evento BOOT_COMPLETED, lo que permite que la aplicación se ejecute automáticamente al encender el dispositivo.

MainService:

Es un servicio en segundo plano, probablemente encargado de mantener la conexión con el servidor de comando y control.

Búsqueda de cadenas en el código smali

Se realizaron búsquedas con grep en la carpeta smali/:

```
grep -r "http" smali/
```

Resultados relevantes:

smali/com/metasploit/stage/b.smali: contiene la cadena "http"

smali/com/metasploit/stage/Payload.smali: contiene la cadena "https"

La presencia de la ruta `com/metasploit/stage/` es una prueba concluyente de que la APK incorpora el payload de Meterpreter de Metasploit. Esta estructura de paquetes es característica de las cargas útiles generadas con `msfvenom`.

Búsqueda adicional con strings

Se ejecutó `strings` para buscar patrones como IPs o puertos típicos:

```
strings malicious.apk | grep -i "meterpreter\\|ahmyth\\|reverse_tcp\\|4444\\|192.168"
```

No se obtuvieron resultados, lo que puede deberse a ofuscación o a que las direcciones IP no están en texto plano. Sin embargo, la evidencia del paquete `com/metasploit/stage` es suficiente.

Conclusiones:

A partir de los resultados obtenidos, se concluye que la aplicación `malicious.apk` es maliciosa. Se trata de un troyano de acceso remoto (RAT) generado con Metasploit (Meterpreter). Las principales evidencias son:

Solicita una amplia gama de permisos peligrosos, lo que excede cualquier funcionalidad legítima.

Declara un receptor para `BOOT_COMPLETED` y un servicio, garantizando persistencia tras el reinicio.

El código `smali` contiene referencias explícitas a `com/metasploit/stage`, correspondiente al payload de Meterpreter.

Este tipo de malware permite a un atacante controlar el dispositivo de forma remota, accediendo a la ubicación, cámara, micrófono, mensajes, contactos y realizando llamadas o enviando SMS sin consentimiento del usuario.

Recomendaciones:

No instalar esta aplicación en ningún dispositivo.

En caso de haberla instalado, realizar un análisis completo con soluciones antimalware y considerar un restablecimiento de fábrica.

Mantener actualizado el sistema operativo y evitar la instalación de aplicaciones de fuentes no oficiales.